Tecnologia em Análise e Desenvolvimento de Sistemas - TADS

Disciplina de Estrutura de Dados

Prof. Luciano Vargas Gonçalves

Atividade Avaliada 1

Data de entrega: 29/03/2026

Entrega em Aula (Valor 2,5 pontos no primeiro bimestre)

1 Implemente um sistema para gerenciar uma festa de aniversário, realize o cadastro de Convidados, Trabalhadores, Convite, Data, Horário e Festa. O sistema deverá contar com as estruturas “STRUCT” abaixo para armazenar as informações:

```
typedef struct{
    int dia;
    int mes;
    int ano;
} Data;

typedef struct{
    int hora;
    int minuto;
} Horario;

typedef struct{
    int codigo;
    char nome[100];
    char telefone[20];
    int confirmou; // 0 = não confirmado, 1 = confirmado
} Convidado;

typedef struct{
    int codigo;
    char nome[100];
    char funcao[50];
    float pagamento;
} Trabalhador;

typedef struct{
    int codigo;
    Convidado *convidado;
    char mensagem[200];
} Convite;

typedef struct{
    char nome[100];
    char local[100];
    Data data;
    Horario horario;
    Convidado convidados[50];
    int totalConvidados;
    Trabalhador trabalhadores[20];
    int totalTrabalhadores;
    Convite *convites;
    int totalConvites;
} Festa;
```

2 Detalhes das Estruturas para armazenar dados de uma festa.

2.a ==========Funções de Cadastro ==========

Função de cadastro de convidado -- int cadastraConvidado(Festa *festa);

Realiza o cadastro de um novo convidado na festa, solicitando os dados via entrada padrão (teclado). O convidado é inserido no vetor de convidados da estrutura Festa

Parâmetros Entrada - Teclado

Festa *festa → ponteiro para a estrutura festa onde o convidado será armazenado.

Parâmetros saída: INT

Retorno:

1 → cadastro realizado com sucesso

0 → erro (ex: código já existente ou limite atingido)

==========

Função de cadastro de trabalhador -- int cadastraTrabalhador(Festa *festa);

Realiza o cadastro de um trabalhador da festa (ex: garçom, DJ, segurança), armazenando suas informações no vetor de trabalhadores.

Parâmetros Entrada - Teclado

Festa *festa → ponteiro para a estrutura festa.

Parâmetros saída: INT

Retorno:

1 → sucesso

0 → erro (ex: limite atingido)

==========

Função cria um convite -- int criarConvite(Festa *festa);

Cria um convite para um convidado já cadastrado. A função deve localizar o convidado pelo código, associá-lo ao convite por meio de um ponteiro e armazenar o convite em uma estrutura alocada dinamicamente.

Parâmetros:

Festa *festa → ponteiro para a estrutura festa.

Retorno: INT

1 → convite criado com sucesso

0 → erro (ex: convidado não encontrado)

==========Funções de Operações ==========

Função confirma presença -- int confirmarPresenca(Festa *festa);

Atualiza o status de confirmação de presença de um convidado. A função deve localizar o convidado pelo código e marcar o campo confirmou como 1.

Parâmetros Entrada:

Festa *festa → ponteiro para a estrutura festa.

Retorno: INT

1 → confirmação realizada

0 → convidado não encontrado

==========

Função calcular total de gastos da festa -- float calcularTotalPagamentos(Festa *festa);

Calcula o valor total a ser pago aos trabalhadores cadastrados na festa, somando o campo pagamento de cada trabalhador.

Parâmetros:

Festa *festa → ponteiro para a estrutura festa.

Retorno: FLOAT

Valor total (tipo float) correspondente à soma dos pagamentos.

========== Funções de Exibição ==========
Função mostra convidado -- void mostraConvidado(Convidado c);
Exibe todas as informações de um convidado (código, nome, telefone e status de confirmação).
Parâmetros:
Convidado c → estrutura passada por valor.
Retorno:
Não possui retorno (void)

==========
Função mostra trabalhador -- void mostraTrabalhador(Trabalhador t);
Exibe os dados de um trabalhador da festa, incluindo sua função e pagamento.
Parâmetros:
Trabalhador t → estrutura passada por valor.
Retorno:
Não possui retorno

==========
Função mostra convite -- void mostraConvite(Convite c);
Exibe os dados de um convite, incluindo a mensagem e os dados do convidado associado (acessado via ponteiro).
Parâmetros:
Convite c → estrutura passada por valor.
Retorno:
Não possui retorno

==========
Função mostra a festa -- void mostraFesta(Festa f);
Exibe todas as informações da festa, incluindo:
dados gerais (nome, local, data, horário)
lista de convidados
lista de trabalhadores
lista de convites
Parâmetros: Festa f → estrutura passada por valor.
Retorno:
Não possui retorno

==========
Função mostra confirmados -- void listarConfirmados(Festa f);
Exibe apenas os convidados que confirmaram presença (confirmou == 1).
Parâmetros:
Festa f → estrutura passada por valor.
Retorno:
Não possui retorno

==========
Função buscar convidado -- int buscarConvidado(Festa *festa, int codigo);
Busca um convidado pelo código dentro do vetor de convidados.
Parâmetros:
Festa *festa → ponteiro para a estrutura festa
int codigo → código do convidado a ser buscado
Retorno:
Índice do convidado no vetor
-1 → não encontrado

==========

Função busca trabalhador -- int buscarTrabalhador(Festa *festa, int codigo);

Busca um trabalhador pelo código no vetor de trabalhadores.

Parâmetros:

Festa *festa

int codigo

Retorno:

Índice no vetor

-1 → não encontrado

==========

Função liberar memória -- void liberarMemoria(Festa *festa);

Libera toda a memória alocada dinamicamente para os convites, evitando vazamentos de memória.

Parâmetros:

Festa *festa → ponteiro para a estrutura festa

Retorno:

Não possui retorno

==========

3 O programa principal deve:

3.a Realizar o pré-cadastrado 1 festa, 6 convidados, 5 trabalhadores, 1 convite. Estas informações deve estar registradas no código.

3.b Para armazenar as reservas de dados utilize alocação dinâmica (Malloc);

3.c No mínimo 2 convites devem ser buscados e para serem confirmados.

3.d Apresente todas as informações da festa registradas;

3.e Crie um menu de opções para chamada das funções:

```
===== MENU FESTA =====
1. Cadastrar Convidado
2. Cadastrar Trabalhador
3. Criar Convite
4. Confirmar Presenca
5. Listar Convidados
6. Listar Trabalhadores
7. Listar Confirmados
8. Mostrar Festa
9. Calcular Total de Pagamentos
0. Sair
Opcao: 1
```

3.f Ao Final do programa todas reservas de memória locadas (malloc) devem ser e liberadas (FREE);

3.g Caso o cadastro de convidado, trabalhador e convite não for possível (retorno 0) apresente uma mensagem informando o erro.

Observação:

O trabalho deve ser enviado a plataforma SIGAA, até o prazo final.
Dúvidas nos horários de atendimentos ou por email;

Resumo dos assuntos abordados

- Trabalhar com structs compostas
- Utilizar ponteiros para manipulação de dados
- Aplicar alocação dinâmica (malloc/free)
- Implementar funções com passagem por referência
- Desenvolver um sistema modular em C

Bom trabalho!